



Document 5: Analysis — Functional Requirements

Prepared by: Eman Al-Aodat | Mobile Application Development Course

1. Functional Requirements Analysis

This document provides a detailed analysis of the functional requirements for LearnHub, including inputs, processing logic, outputs, and implementation notes for each major system feature.

1.1 FR-01 & FR-02: Splash Screen & Onboarding

Input	App launch event
Processing	Check SharedPreferences for 'first_launch' flag; if true show 3-step dialog and set flag to false.
Output	Splash animation for 2s, then conditional onboarding or home screen.
Implementation	SharedPreferences bool flag 'onboarding_shown'; SplashScreen widget with Future.delayed(Duration(seconds:2)).
Edge Case	If onboarding is dismissed early, flag is still set to prevent re-showing.

1.2 FR-03, FR-04, FR-05: Course Listing, Search & Filter

Input	Course data (static list of 8 courses); user search query string; selected category chip.
Processing	Filter course list by: (1) matching search query in course title (case-insensitive), AND (2) selected category. Both filters apply simultaneously.
Output	Filtered ListView of CourseCard widgets displaying title, category, rating, and instructor.
Implementation	CoursesProvider exposes filteredCourses getter; uses String.toLowerCase().contains() for search; category filter uses == comparison.
Edge Case	If no courses match, a 'No courses found' empty state widget is shown.

1.3 FR-06: Course Detail Screen

Input	Course object passed via Navigator.push() as route argument.
Processing	Reads enrollment status and progress value from CoursesProvider for the given course ID.
Output	Screen displaying: course image, title, category chip, rating, instructor, lesson count, duration, description, progress slider, and enroll/unenroll button.
Implementation	CourseDetailScreen(course: course) receives Course model; context.watch<CoursesProvider>() for reactive updates.
Edge Case	Progress slider only active if course is enrolled; shows 'Drag slider to update progress' hint.

1.4 FR-07: Enrollment System

Input	Course ID; user tap on Enroll/Unenroll button.
Processing	CoursesProvider.toggleEnrollment(id): if ID in _enrolledIds set → remove; else → add. Calls _saveToPrefs() immediately.
Output	Button text changes between 'Enroll' and 'Unenroll'; enrollment badge appears/disappears on course card; My Courses screen updates.
Implementation	_enrolledIds: Set<String> in CoursesProvider; persisted as List<String> via SharedPreferences.setStringList().
Edge Case	Unenrolling resets progress to 0% for that course.

1.5 FR-08: Progress Tracking

Input	Slider value (double 0.0 to 100.0); course ID.
Processing	CoursesProvider.updateProgress(id, value): updates _progressMap[id] = value; calls _saveToPrefs() to persist.
Output	Slider position updates in real time; progress bar in My Courses updates; stats (average progress, completed count) recalculate.
Implementation	_progressMap: Map<String, double>; persisted via SharedPreferences as JSON-encoded map; 100% marks course as completed.
Edge Case	Progress resets to 0 on unenrollment.

1.6 FR-09: My Courses Screen

Input	Enrolled course IDs and progress values from CoursesProvider.
Processing	Filters full course list to enrolled only; calculates average progress (sum/count); counts courses at 100%.
Output	Summary card (enrolled count, average %, completed count); list of enrolled courses with linear progress bar and % label.
Implementation	MyCoursesScreen reads provider.enrolledCourses and provider.getProgress(id) for each course.
Edge Case	If no courses enrolled, displays empty state illustration and 'Enroll in a course to get started' message.

1.7 FR-10 to FR-13: Profile, Dark Mode & Data Management

FR	Feature	Analysis
FR-10	Profile Editing	TextFormField inside Form widget; GlobalKey<FormState> triggers

		validation. Validators: not empty + min 2 chars. On save: <code>provider.setUsername() + SharedPreferences.setString('username')</code> .
FR-11	Dark Mode	Boolean <code>isDarkMode</code> in <code>CoursesProvider</code> . <code>MaterialApp</code> reads via <code>context.watch()</code> to switch <code>ThemeMode</code> . Persisted via <code>SharedPreferences.setBool('dark_mode')</code> .
FR-12	Data Persistence	<code>loadFromPrefs()</code> called before <code>runApp()</code> at startup. All state (<code>enrolledIds</code> , <code>progressMap</code> , <code>isDarkMode</code> , <code>username</code>) loaded once and held in <code>CoursesProvider</code> memory. Saved after every state mutation.
FR-13	Clear All Data	<code>prefs.clear()</code> resets all keys. Provider state reset to defaults. App navigates back to home. Requires confirmation dialog before execution.